

Asignacion de Modulos - Equipo de Datos

Miembros del equipo

Miembro	Rol	Modulo asignado
Allan	Backend Developer	Prisma ORM + Migraciones + Seed
Yandri	Backend Developer	Repositorios Infraestructura (implementaciones)
Johnny	Database Architect	Modelado de datos + Interfaces de repositorios
Ricardo	Full Stack Developer	Supabase Storage + Upload de imagenes
Kelvin	DevOps / QA	Tests de repositorios + Integracion continua

Allan — Prisma ORM + Migraciones + Seed

Responsabilidades

- Configuracion inicial de Prisma 7 con adapter PostgreSQL (**PrismaPg**)
- Creacion y mantenimiento del archivo **prisma/schema.prisma**
- Generacion del Cliente Prisma (**prisma generate**)
- Creacion y aplicacion de migraciones (**prisma migrate dev**)
- Poblacion de datos de prueba (**prisma/seed.ts**)

Archivos a cargo

```
prisma/
├── schema.prisma      ← Modelos de datos
├── seed.ts            ← Datos de prueba
├── migrations/         ← Historial de migraciones
└── config.ts           ← Configuracion Prisma v7

src/infrastructure/database/
└── prisma.service.ts   ← Singleton de conexion
```

Tareas completadas

1. Configurar Prisma 7 con **@prisma/adapters-pg** + **pg.Pool**
 2. Modelar 6 tablas: Empleado, Usuario, Vehiculo, PlanFinanciamiento, Cotizacion, Venta
 3. Ejecutar 5 migraciones: **init**, **update_vehiculo_plan**, **add_cotizacion_venta**, **add_vehiculo_imagen**, **add_cliente_cedula_ciudad**
 4. Poblar DB con 4 usuarios, 5 empleados, 8 vehiculos, 4 planes, 1 cotizacion, 1 venta
 5. Implementar **prisma.service.ts** con conexion pool + singleton
-

Yandri — Repositorios Infraestructura

Responsabilidades

- Implementar TODAS las interfaces de repositorio definidas por Johnny
- Escribir consultas Prisma para cada operacion CRUD
- Mapear entre filas de DB y entidades del dominio (`desdeDatos()` / `toJSON()`)
- Garantizar que cada repositorio solo dependa de `getPrisma()` y entidades

Archivos a cargo

```
src/infrastructure/repositories/  
├─ empleado.supabase.repository.ts  
├─ usuario.supabase.repository.ts  
├─ vehiculo.supabase.repository.ts  
├─ plan.supabase.repository.ts  
├─ cotizacion.supabase.repository.ts  
└─ venta.supabase.repository.ts
```

Tareas completadas

1. `EmpleadoSupabaseRepository` — CRUD + `findAll`, `count`
2. `UsuarioSupabaseRepository` — CRUD + `findByUsuario`, `findByRol`, `count`
3. `VehiculoSupabaseRepository` — CRUD + `findByTipo`, `findDisponibles`, `findActivos`, `count`
4. `PlanFinanciamientoSupabaseRepository` — CRUD + `findActivos`, `count`
5. `CotizacionSupabaseRepository` — CRUD + `findActivas`, `findByVehiculo`, `count`
6. `VentaSupabaseRepository` — CRUD + `findByAsesor`, `findVentasPorPeriodo`, `findTopVehiculos`, `count`

Patron usado en cada implementacion

```
async findById(id: string): Promise<Entidad | null> {  
  const prisma = getPrisma();  
  const data = await prisma.modelo.findUnique({ where: { id } });  
  return data ? Entidad.desdeDatos(data) : null;  
}
```

Johnny — Modelado + Interfaces de Repositorio

Responsabilidades

- Diseñar el esquema de datos (tablas, columnas, tipos, relaciones)
- Definir las interfaces (contratos) que deben implementar los repositorios
- Establecer las reglas de validacion y constraints a nivel de DB
- Asegurar que el diseno soporte los requisitos del negocio

Archivos a cargo

```
prisma/schema.prisma          ← Diseño de modelos (en conjunto con Allan)

src/domain/repositories/
├─ empleado.repository.ts
├─ usuario.repository.ts
├─ IVehiculoRepository.ts
├─ IPlanFinanciamientoRepository.ts
├─ ICotizacionRepository.ts
├─ IVentaRepository.ts
└─ IAsesorRepository.ts      ← No implementado (pendiente)
```

Principios aplicados

- **Arquitectura Hexagonal:** las interfaces estan en **domain/**, las implementaciones en **infrastructure/**
- **Metodos minimos:** cada interface define solo los metodos que el negocio necesita
- **Tipado fuerte:** metodos devuelven entidades del dominio, no tipos de Prisma

Ejemplo de interface diseñada por Johnny

```
export interface CotizacionRepository {
  save(cotizacion: Cotizacion): Promise<void>;
  update(cotizacion: Cotizacion): Promise<void>;
  delete(id: string): Promise<boolean>;
  findById(id: string): Promise<Cotizacion | null>;
  findAll(): Promise<Cotizacion[]>;
  findActivas(): Promise<Cotizacion[]>;
  findByVehiculo(vehiculoId: string): Promise<Cotizacion[]>;
  count(): Promise<number>;
}
```

Ricardo — Supabase Storage + Upload

Responsabilidades

- Configurar Supabase Storage (bucket **vehiculos**)
- Implementar subida de archivos desde el frontend
- Configurar RLS policies para acceso publico
- Integrar upload con el modulo de Vehiculos
- Implementar **UploadController** y ruta **/api/upload**

Archivos a cargo

```
src/infrastructure/storage/
└─ supabase.storage.ts      ← Cliente Supabase + upload/delete

src/interfaces/controllers/
```

```

└─ UploadController.ts      ← Endpoint POST /api/upload

src/interfaces/routes/
└─ uploadRoutes.ts          ← Wire de ruta

frontend/src/components/
└─ VehiculoForm.tsx         ← Input file con preview
    
```

Flujo implementado por Ricardo

```

Usuario selecciona imagen en VehiculoForm
↓
Petición POST /api/upload (multipart)
↓
UploadController recibe el archivo
↓
supabase.storage.uploadFile()
↓
Supabase Storage guarda en bucket "vehiculos"
↓
Retorna URL publica
↓
Se guarda URL en campo `imagen` del Vehiculo (via PUT /api/vehiculos/:id)
    
```

Configuracion de RLS

3 policies creadas en Supabase Dashboard para el bucket **vehiculos**:

Policy	Operacion	Target
give anon access to INSERT	INSERT	anon
give anon access to SELECT	SELECT	anon
give anon access to DELETE	DELETE	anon

Kelvin — Tests de Repositorios + Integracion

Responsabilidades

- Escribir tests unitarios y de integracion para cada repositorio
- Verificar que CRUD funciona contra BD real
- Validar que los metodos de busqueda devuelven datos correctos
- Integrar tests en el pipeline (scripts npm)

Archivos a cargo

```

tests/
├── test-empleado-repository.ts
├── test-usuario-repository.ts
├── test-vehiculo-repository.ts
├── test-cotizacion-entity.ts      ← Tests de entidad + repositorio
├── test-venta-entity.ts
├── test-empleado-entity.ts
├── test-usuario-entity.ts
├── test-plan-entity.ts
├── test-vehiculo.ts
├── test-empleado-service.ts
└── test-usuario-service.ts

```

Tareas completadas

1. Test de **UsuarioSupabaseRepository** — CRUD completo + búsquedas por rol
2. Test de **EmpleadoSupabaseRepository** — CRUD completo + conteo
3. Test de **VehiculoSupabaseRepository** — CRUD + filtros por tipo/disponibilidad
4. Test de entidades (Cotizacion, Venta, Usuario, Empleado, Plan, Vehiculo)
5. Tests de servicios (Usuario, Empleado)
6. Script `npm run test:all` que ejecuta toda la suite (21+ pruebas)

Cobertura de pruebas

```

npm run test:all           # Suite completa
npm run test:cotizaciones  # Tests modulo cotizacion
npm run test:repository-usuario # Test especifico de repositorio

```

Resumen de dependencias entre miembros

```

Johnny (interfaces)
  ↓ define contratos
Yandri (implementaciones)
  ↓ usa
Allan (Prisma ORM + conexion)
  ↓ conecta a
Supabase PostgreSQL
  ↑
Ricardo (Storage para imagenes)
  ↑
Kelvin (prueba todo con tests)

```

Nota: El modulo **AsesorRepositoryImpl.ts** y sus interfaces asociadas (**IAAsesorRepository.ts**) estan creados pero **no asignados** — quedaron como caso de estudio pendiente para el equipo.